

UCS503 - SOFTWARE ENGINEERING

Workload Balancer

Personalized Student Workload Management and Study Planning System

Submitted by:

1024030800 Shaurya Garg
1024030801 Vriti Mahajan
1024030805 Umang Gautam
1024030809 Khushi Chhabra

Group: 1 BE Third Year, CSE

Submitted to:

NISHA THAKUR

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	3
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	5
2.1.3	System Design Diagrams	6
2.2	Implementation Details	11
2.2.1	Module 1: Academic Data Management	11
2.2.2	Module 2: Secondary Features	12
3	Results and Evaluation	13
3.1	Testing Methodology	13
3.2	Performance Metrics	13
3.3	Analysis	13
4	Conclusion	14
4.1	Summary of Work	14
4.2	Key Findings	14
4.3	Future Work and Recommendations	14
4.4	Final Remarks	14
	Bibliography	15

Introduction

1.1 Background and Context

Students manage several academic responsibilities at once: subjects, topics, assignments, performance, study sessions, and deadlines. Planning this by hand makes it hard to decide what to study first and how to divide the available time.

The **Workload Balancer** is a personalized student workload management and study planning system. It records a student's academic workload, calculates priorities, generates a study plan, tracks study sessions, and rebalances the plan when things change.

1.1.1 Problem Statement

When many assignments, topics, and deadlines occur together, it is difficult to judge the priority of each activity and distribute study time effectively. Manual plans also do not adapt: a missed session or a shifted deadline can invalidate the whole schedule.

The Workload Balancer addresses this by providing a single system that records the workload, calculates priority, generates a personalized plan, and rebalances it when changes occur.

1.2 Project Objectives

The main objectives of the Workload Balancer are:

1. **Manage Subjects and Topics:** add, edit, delete, and view subjects and their topics.
2. **Manage Assignments:** record assignments and deadlines.
3. **Record Performance:** store scores and derive topic mastery.
4. **Calculate Priority Scores:** compute priority from mastery and urgency.
5. **Generate a Study Plan:** turn priorities into a schedule.
6. **Manage Study Sessions:** view sessions and mark them completed or missed.
7. **Rebalance the Plan:** detect changes, re-score, and re-plan when required.

Methodology and Design

2.1 System Architecture

The system follows the flow of student information through workload processing, priority calculation, plan generation, session tracking, and rebalancing:

Student Data → Priority Calculation → Study Plan Generation → Session Tracking → Rebalancing

The backend is organized as a layered architecture in which each layer calls only the one below it: routes handle requests, services hold the business logic, and repositories access the data. This separation keeps each part independently understandable and testable, and lets the rebalancing workflow act only through the service layer.

System Architecture

Layered backend — each layer calls only the one below it — with the frontend on top and the rebalancing agent restricted to the service layer.

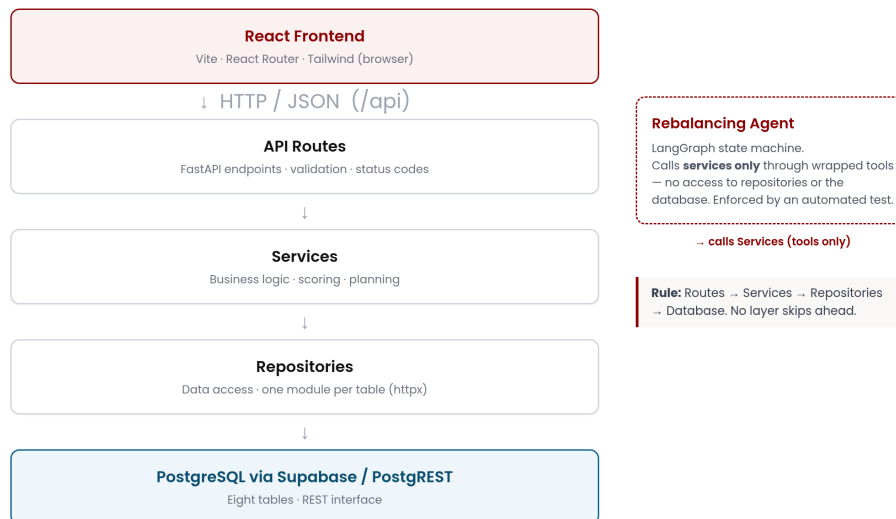


Figure 2.1: High-Level Architecture of the Workload Balancer

2.1.1 High-Level Design

The student provides academic information such as subjects, topics, deadlines, and performance. The system calculates a priority for each topic from mastery and urgency, and uses these to generate a study plan. Sessions can be marked completed or

missed; when a relevant change occurs, the system re-scores the topics, re-plans the remaining sessions, and presents the updated plan.



Figure 2.2: Overall Activity Flow of the Workload Balancer

2.1.2 Technical Stack

- **Frontend:** React [Met13] with Vite [You20] and Tailwind CSS [Tai17].
- **Backend:** Python [Pyt24] and FastAPI [Ram18], with Pydantic [Col19] for validation.

- **Database:** PostgreSQL [The24].
- **AI / Workflow:** LangGraph [Lan24] and an LLM-based workflow for intelligent study-plan generation and rebalancing.
- **LLM Access:** OpenRouter [Ope23] for language-model access and structured outputs.
- **Development Tools:** Visual Studio Code, Git, GitHub, and draw.io.

2.1.3 System Design Diagrams

The system was modelled using standard UML and design diagrams before and during implementation. The use-case diagram captures what the student can do; the class diagram captures the data model; the data-flow and sequence diagrams capture how information moves through the system; and the swimlane diagram shows responsibilities across the student, the system, and the database.

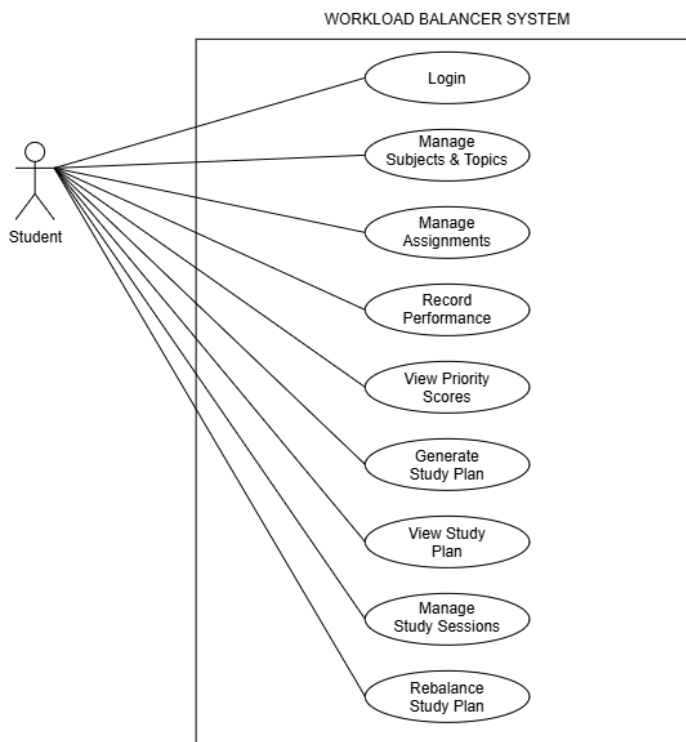


Figure 2.3: Overall Use Case Diagram

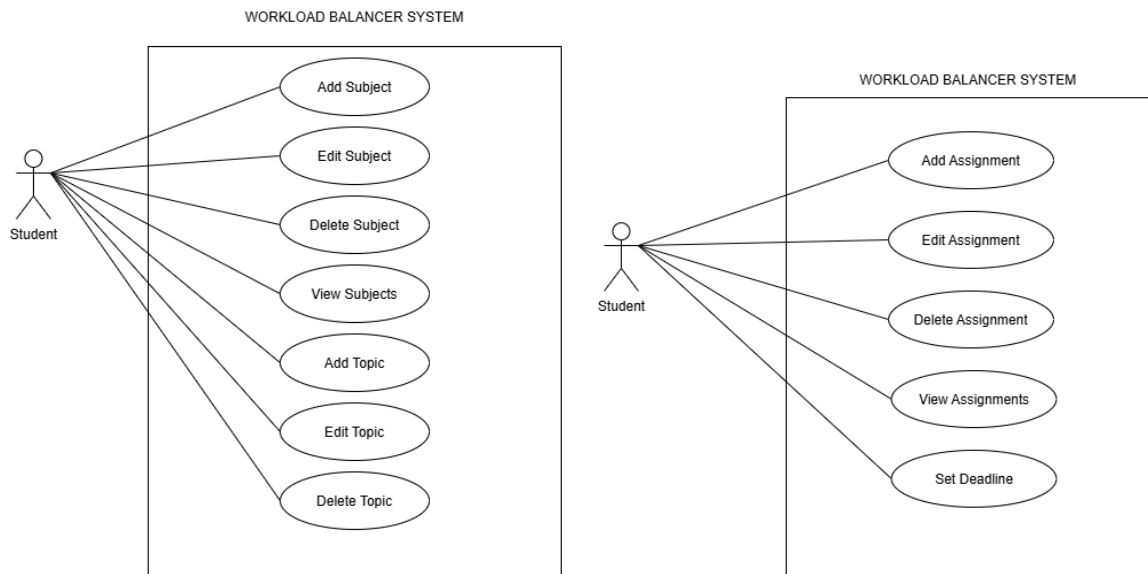


Figure 2.4: Use Cases: Manage Subjects & Topics (left), Manage Assignments (right)

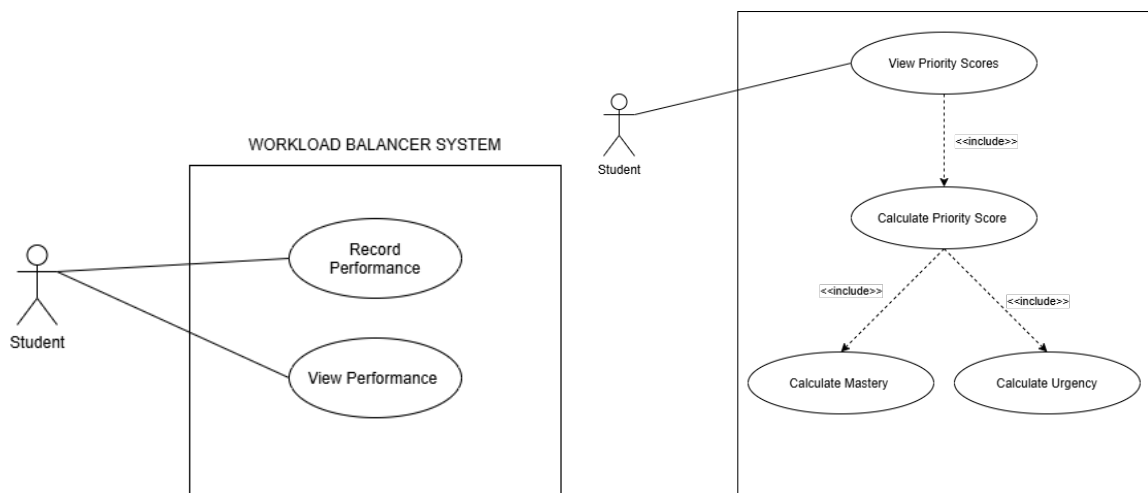


Figure 2.5: Use Cases: Record Performance (left), Calculate Priority Scores (right)

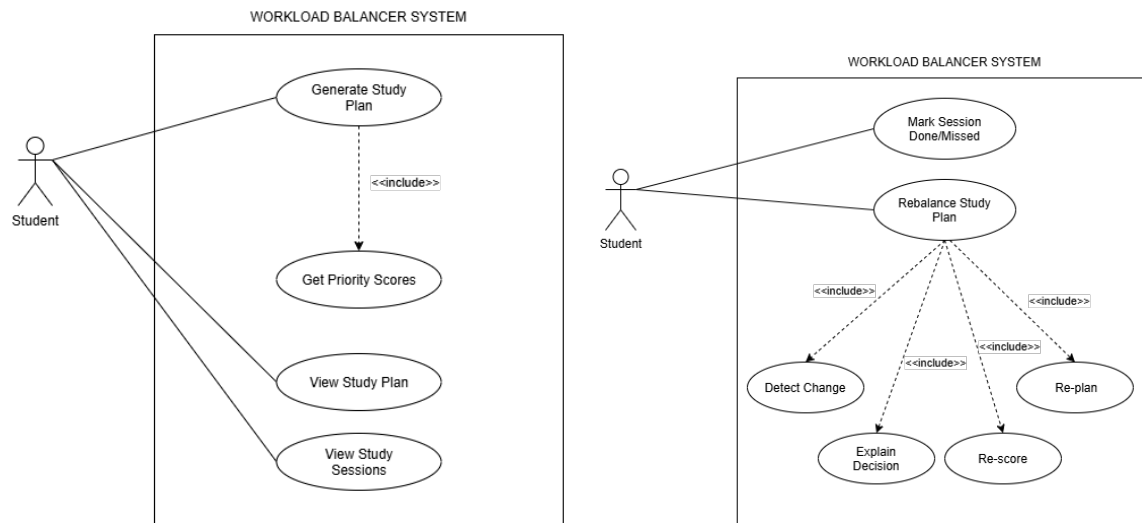


Figure 2.6: Use Cases: Generate and View Study Plan (left), Adaptive Rebalancing (right)

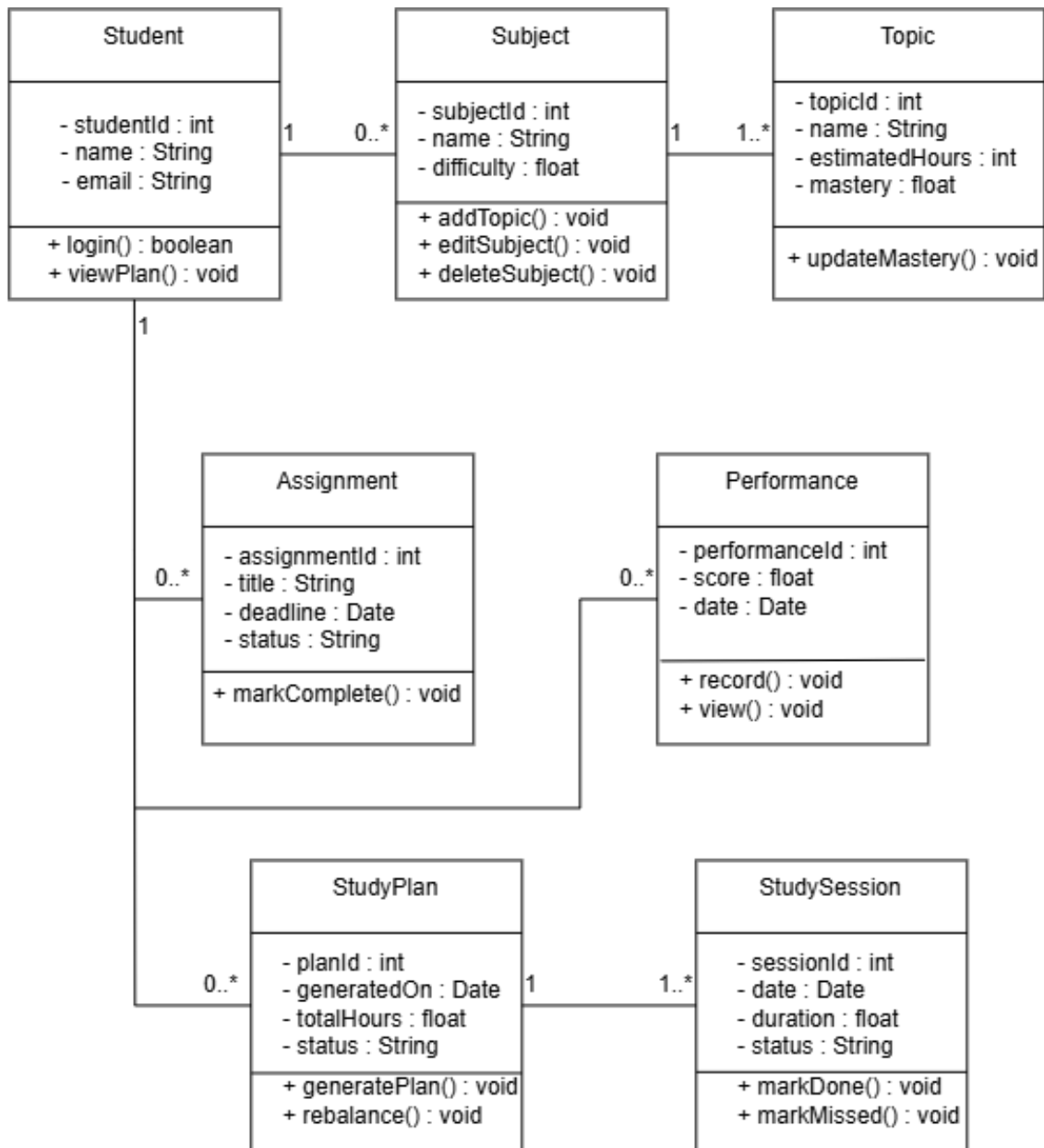


Figure 2.7: Class Diagram

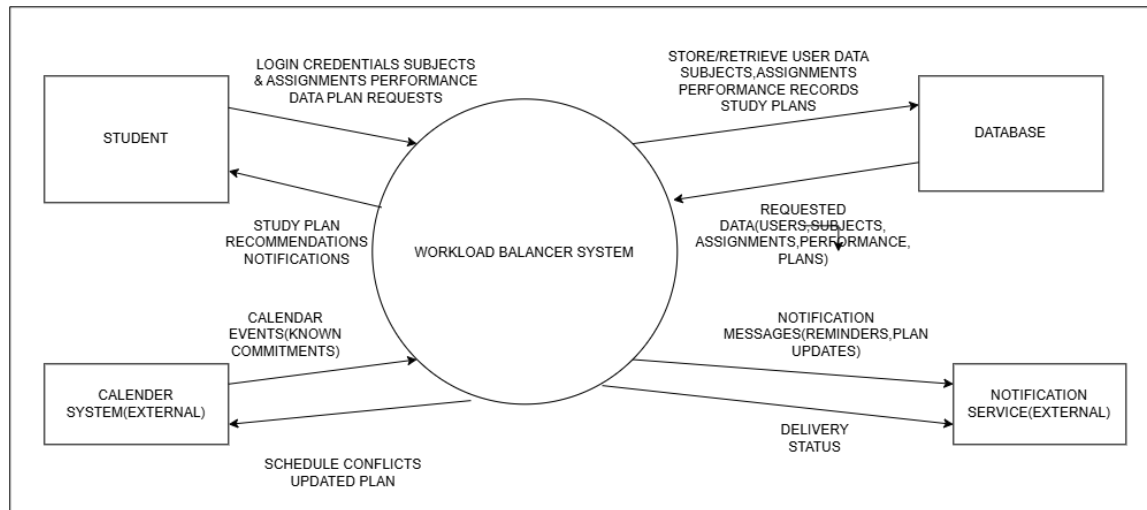


Figure 2.8: Data Flow Diagram

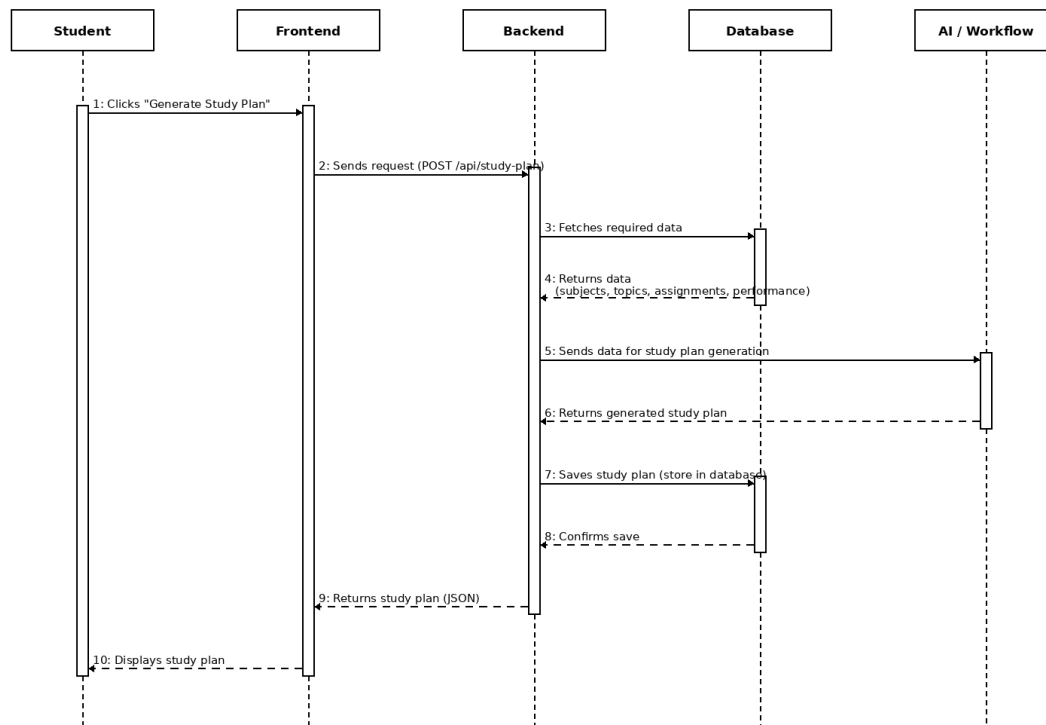


Figure 2.9: Sequence Diagram: Study Plan Generation

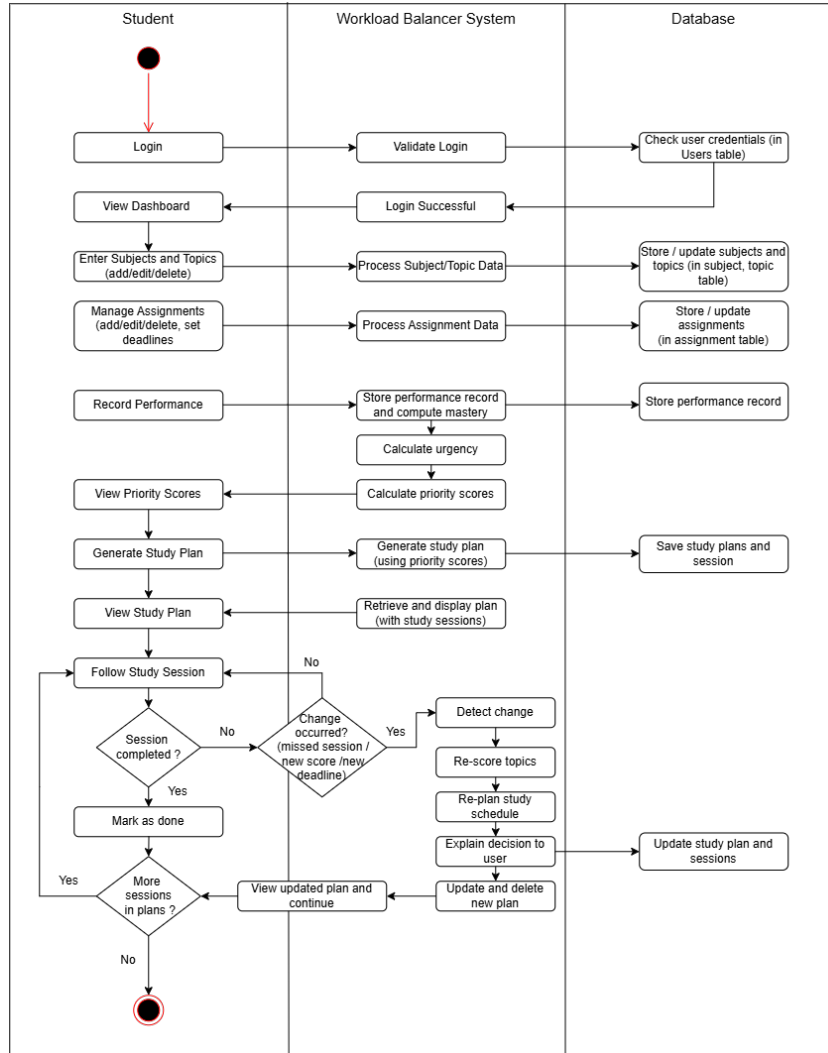


Figure 2.10: Swimlane Diagram: Student, System, and Database Responsibilities

2.2 Implementation Details

2.2.1 Module 1: Academic Data Management

This module maintains the data that all planning depends on: students, the subjects they are enrolled in, the topics within each subject, the deadlines attached to those topics, and the performance scores recorded per topic. It provides full create, view, and delete operations for each, with input validated before it is stored. Subjects and topics are shared, and students connect to them through enrollments, so shared content is not duplicated per student.

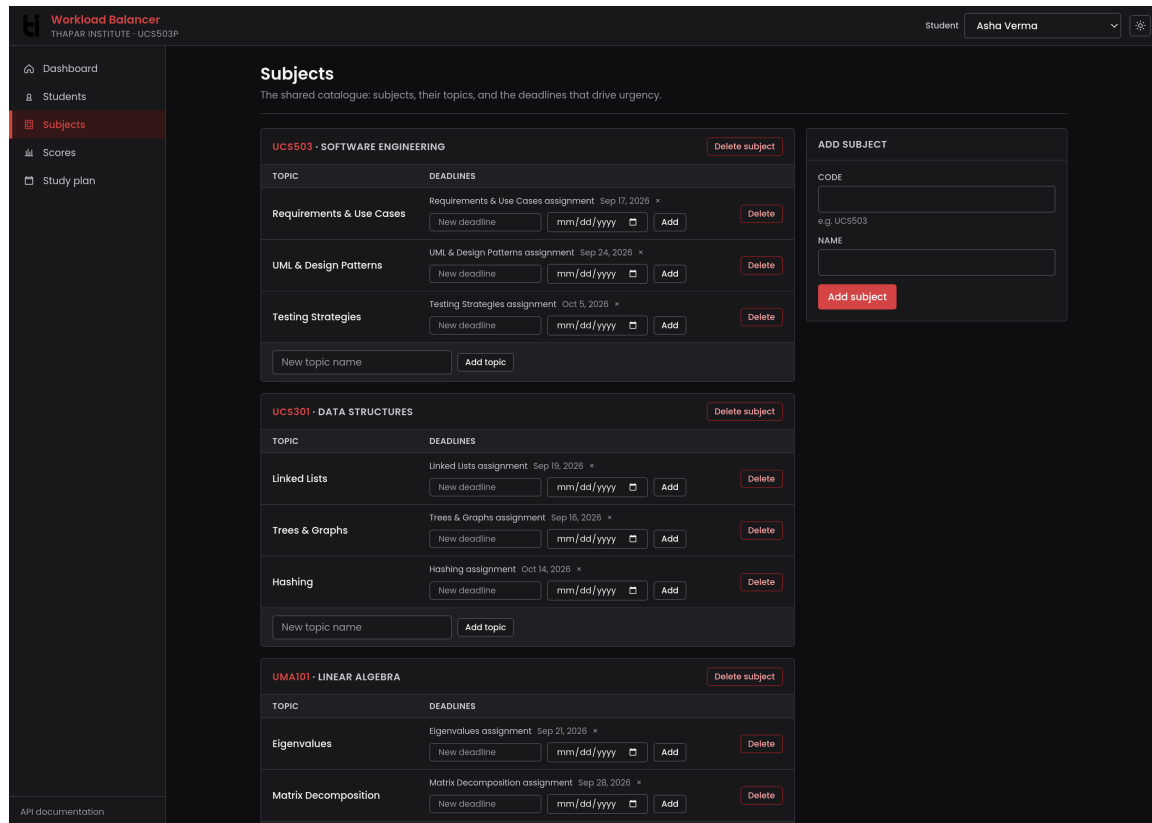


Figure 2.11: Subjects, Topics, and Deadlines Management Interface

2.2.2 Module 2: Secondary Features

Continue documenting your implementation by module, demonstrating the complexity and depth of your work.

Results and Evaluation

3.1 Testing Methodology

Describe your testing strategy:

- **Unit Testing:** How you tested individual components.
- **Integration Testing:** How you verified components work together.
- **Performance Testing:** How you measured system performance.
- **User Testing:** If applicable, how users evaluated your system.

3.2 Performance Metrics

Present your results clearly with tables and figures. For example:

Metric	Target	Achieved
Response Time	< 100ms	85ms
Accuracy	> 95%	97.2%
Throughput	1000 req/s	1250 req/s

Table 3.1: Performance Results Summary

3.3 Analysis

Analyze your results:

- Did you meet your objectives?
- Where did you exceed expectations?
- What challenges did you encounter?
- How did you resolve them?

Conclusion

4.1 Summary of Work

Summarize what you accomplished. Remind readers of your objectives and explain which ones you met.

4.2 Key Findings

- **Finding 1:** State a significant result or insight from your work.
- **Finding 2:** Describe another important discovery.
- **Finding 3:** Highlight any unexpected results.

4.3 Future Work and Recommendations

Discuss how others could extend or improve your work:

1. Potential enhancements to your system [latex2e]
2. Alternative approaches worth exploring
3. Scalability improvements
4. Integration with other systems
5. Real-world deployment considerations

4.4 Final Remarks

Conclude with reflections on what you learned and the impact of your work.

Bibliography

- [Col19] Samuel Colvin. *Pydantic: Data Validation Using Python Type Hints*. 2019 (cit. on p. 5).
- [Lan24] LangChain, Inc. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. LangChain, Inc. 2024 (cit. on p. 6).
- [Met13] Meta Open Source. *React: The Library for Web and Native User Interfaces*. Meta Platforms, Inc. 2013 (cit. on p. 5).
- [Ope23] OpenRouter, Inc. *OpenRouter: A Unified Interface for Large Language Models*. OpenRouter, Inc. 2023 (cit. on p. 6).
- [Pyt24] Python Software Foundation. *Python Language Reference*. Python Software Foundation. 2024 (cit. on p. 5).
- [Ram18] Sebastián Ramírez. *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. 2018 (cit. on p. 5).
- [Tai17] Tailwind Labs. *Tailwind CSS: A Utility-First CSS Framework*. Tailwind Labs Inc. 2017 (cit. on p. 5).
- [The24] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. The PostgreSQL Global Development Group. 2024 (cit. on p. 6).
- [You20] Evan You. *Vite: Next Generation Frontend Tooling*. 2020 (cit. on p. 5).